

BCSE204L Design and Analysis of Algorithms

Module-1

Topic-4: Huffman Coding (Greedy Technique)

Dr Sunil Kumar P V

SCOPE

Data Compression Requirements

- Storage and communication
- Huffman codes compress data very effectively: savings of 20% to 90% are typical
- Greedy on the frequency of each character in the data
- Fixed length and variable length coding

Fixed vs Variable length Coding

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

- Fixed length $\rightarrow (45+13+12+16+9+5)*1000*3 = 300000$ bits
- Variable length $\rightarrow$
 $(45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4)*1000 = 224000$ bits
- Saving 26%

Prefix Codes

- Codes in which no codeword is also a prefix of some other codeword are called **prefix codes**
- Advantage:- decoding without ambiguity
- Prefix code can always achieve the optimal data compression among any character coding scheme

Prefix Codes Example

- Encoding:- $abc \rightarrow 0 \cdot 101 \cdot 100$ where $\cdot$ is concatenation
- Therefore, instead of abc we send 0101100
- Decoded at the receiving end without ambiguity, provided the receiver must know the coding table

Huffman Coding

- Produces optimal prefix codes based on the frequency of occurrence of characters within the message/data
- Follows **greedy strategy**
- Assume that $\mathcal{C}$ is a set of n characters and that each
- character $c \in \mathcal{C}$ is an object with an attribute $c.freq$ giving its frequency

Huffman Tree

- Algorithm builds Huffman tree, T in a bottom up manner
- Initially a set of $|C|$ leaves and performs $|C| - 1$ merge operations
- Uses a min-priority queue Q , keyed on the *freq* attribute

Huffman Coding Pseudocode

HUFFMAN(C)

```
1   $n = |C|$ 
2   $Q = C$ 
3  for  $i = 1$  to  $n - 1$ 
4      allocate a new node  $z$ 
5       $z.left = x = \text{EXTRACT-MIN}(Q)$ 
6       $z.right = y = \text{EXTRACT-MIN}(Q)$ 
7       $z.freq = x.freq + y.freq$ 
8      INSERT( $Q, z$ )
9  return EXTRACT-MIN( $Q$ )    // return the root of the tree
```


Huffman Coding

- Demo → on board
- Complexity analysis → $O(n \log n)$, if the priority queue is implemented using heap

Exercise

- Draw the Huffman tree and generate the Huffman code

a	e	i	o	u	s	t
10	15	12	3	4	13	1

- Answer: a=111, e=10, i=00, o=11001, u=1101, s=01, t=11000

Thank you..